Mudcard

- **I still don't quite understand the purpose of a decision boundry.**
  - The decision boundary is a line/curve/surface that separates different classes in the feature space
  - So on one side of a binary decision boundary, the model would predict class 0, on the other side it would predict class 1
  - It defines the region where the model predicts the critial probability (often set to 50%).
  - In binary logistic regression, the decision boundary is linear. For other models, it is often a complex, non-linear curve or surface.

Missing values, part 1

By the end of this lecture, you will be able to

- evaluate simple approaches to handle missing values: drop rows and columns
- handle missing values in categorical and ordinal features in preprocessing
- apply imputation techniques on continuous features and evaluate drawbacks

By the end of this lecture, you will be able to

- **evaluate simple approaches to handle missing values: drop rows and columns**
- handle missing values in categorical and ordinal features in preprocessing
- apply imputation techniques on continuous features and evaluate drawbacks

Dataset

- kaggle house price dataset
- check out the train.csv and the dataset description in the `data` folder (`data/data_description.txt`)!

In [1]:
```
# read the data
import pandas as pd
import numpy  as np
from sklearn.model_selection import train_test_split

# Let's load the data
df = pd.read_csv('../data/train.csv')
# drop the ID
df.drop(columns=['Id'],inplace=True)
```

```
# the target variable
y = df['SalePrice']
df.drop(columns=['SalePrice'],inplace=True)
# the unprocessed feature matrix
X = df
print(X.shape)
# the feature names
ftrs = df.columns
print(df.head())
```

```
(1460, 79)
   MSSubClass MSZoning  LotFrontage  LotArea Street Alley LotShape  \
0          60       RL         65.0     8450   Pave   NaN      Reg
1          20       RL         80.0     9600   Pave   NaN      Reg
2          60       RL         68.0    11250   Pave   NaN      IR1
3          70       RL         60.0     9550   Pave   NaN      IR1
4          60       RL         84.0    14260   Pave   NaN      IR1

  LandContour Utilities LotConfig  ... ScreenPorch PoolArea PoolQC Fence  \
0         Lvl    AllPub    Inside  ...           0        0    NaN   NaN
1         Lvl    AllPub       FR2  ...           0        0    NaN   NaN
2         Lvl    AllPub    Inside  ...           0        0    NaN   NaN
3         Lvl    AllPub    Corner  ...           0        0    NaN   NaN
4         Lvl    AllPub       FR2  ...           0        0    NaN   NaN

  MiscFeature MiscVal  MoSold  YrSold  SaleType  SaleCondition
0         NaN       0       2    2008        WD         Normal
1         NaN       0       5    2007        WD         Normal
2         NaN       0       9    2008        WD         Normal
3         NaN       0       2    2006        WD        Abnorml
4         NaN       0      12    2008        WD         Normal

[5 rows x 79 columns]
```

In [2]:
```
print('data dimensions:',df.shape)
perc_missing_per_ftr = df.isnull().sum(axis=0)/df.shape[0]
print('fraction of missing values in features:')
print(perc_missing_per_ftr[perc_missing_per_ftr > 0])
print('data types of the features with missing values:')
print(df[perc_missing_per_ftr[perc_missing_per_ftr > 0].index].dtypes)
frac_missing = sum(df.isnull().sum(axis=1)!=0)/df.shape[0]
print('fraction of points with missing values:',frac_missing)
```

```
data dimensions: (1460, 79)
fraction of missing values in features:
LotFrontage     0.177397
Alley           0.937671
MasVnrType      0.597260
MasVnrArea      0.005479
BsmtQual        0.025342
BsmtCond        0.025342
BsmtExposure    0.026027
BsmtFinType1    0.025342
BsmtFinType2    0.026027
Electrical      0.000685
FireplaceQu     0.472603
GarageType      0.055479
GarageYrBlt     0.055479
GarageFinish    0.055479
GarageQual      0.055479
GarageCond      0.055479
PoolQC          0.995205
Fence           0.807534
MiscFeature     0.963014
dtype: float64
data types of the features with missing values:
LotFrontage     float64
Alley               str
MasVnrType          str
MasVnrArea      float64
BsmtQual            str
BsmtCond            str
BsmtExposure        str
BsmtFinType1        str
BsmtFinType2        str
Electrical          str
FireplaceQu         str
GarageType          str
GarageYrBlt     float64
GarageFinish        str
GarageQual          str
GarageCond          str
PoolQC              str
Fence               str
MiscFeature         str
dtype: object
fraction of points with missing values: 1.0
```

Exclude points or features with missing values

- easy to do with pandas
- if missing values were encountered during data collection, it is likely missing values will occur during deployment too
  - what will you do during deployment?
  - by dropping columns/rows, you basically ignore the missing values

    - is it OK to not predict for a datapoint with missing values when the model is deployed?
        - in finance and medical problems, this is not a luxury you will have
- it's OK to temporarily drop a small fraction of rows/columns to quickly train a model and see if the project is feasible
- but if the project makes it to deployment, you will not be able to ignore the issue

Drop points or features with missing values

- not OK for the house price dataset because all points contain some NaNs.

In [3]:
```
print(df.shape)
# by default, rows/points are dropped
df_r = df.dropna()
print(df_r.shape)
# drop features with missing values
df_c = df.dropna(axis=1)
print(df_c.shape)
```

```
(1460, 79)
(0, 79)
(1460, 60)
```

By the end of this lecture, you will be able to

- evaluate simple approaches to handle missing values: drop rows and columns
- **handle missing values in categorical and ordinal features in preprocessing**
- apply imputation techniques on continuous features and evaluate drawbacks

Categorical or ordinal features: treat missing values as another category

- the BEST thing you can do!
- already covered in the preprocessing lecture (one hot encoding)
- example: missing values in gender
    - if survey only has options for male/female, missing values are likely because those people are outside the gender binary
    - it is a bad idea to impute (try to guess male or female and thus boxing them into the binary)
- example: native country in the adult data
    - missing data are represented as `?`
    - a one-hot encoded feature was assigned to the missing category

In [4]:
```
# read the data
import pandas as pd
import numpy  as np
from sklearn.model_selection import train_test_split
```

```
# Let's load the data
df = pd.read_csv('../data/train.csv')
# drop the ID
df.drop(columns=['Id'],inplace=True)

# the target variable
y = df['SalePrice']
df.drop(columns=['SalePrice'],inplace=True)
# the unprocessed feature matrix
X = df.values
print(X.shape)
# the feature names
ftrs = df.columns
```

```
(1460, 79)
```

In [5]:
```
random_state = 42

# let's split to train, CV, and test
X_train, X_other, y_train, y_other = train_test_split(df, y, train_size=0.6,
X_CV, X_test, y_CV, y_test = train_test_split(X_other, y_other, test_size=0.

print(X_train.shape)
print(X_CV.shape)
print(X_test.shape)
```

```
(876, 79)
(292, 79)
(292, 79)
```

In [6]:
```
# collect the various features
cat_ftrs = ['MSZoning','Street','Alley','LandContour','LotConfig','Neighborh
            'BldgType','HouseStyle','RoofStyle','RoofMatl','Exterior1st','Ex
           'Heating','CentralAir','Electrical','GarageType','PavedDrive','Mi
ordinal_ftrs = ['LotShape','Utilities','LandSlope','ExterQual','ExterCond','
               'BsmtFinType1','BsmtFinType2','HeatingQC','KitchenQual','Func
               'GarageQual','GarageCond','PoolQC','Fence']
ordinal_cats = [['Reg','IR1','IR2','IR3'],['AllPub','NoSewr','NoSeWa','ELO']
               ['Po','Fa','TA','Gd','Ex'],['Po','Fa','TA','Gd','Ex'],['NA','
               ['NA','Po','Fa','TA','Gd','Ex'],['NA','No','Mn','Av','Gd'],['
               ['NA','Unf','LwQ','Rec','BLQ','ALQ','GLQ'],['Po','Fa','TA','G
               ['Sal','Sev','Maj2','Maj1','Mod','Min2','Min1','Typ'],['NA','
               ['NA','Unf','RFn','Fin'],['NA','Po','Fa','TA','Gd','Ex'],['NA
               ['NA','Fa','TA','Gd','Ex'],['NA','MnWw','GdWo','MnPrv','GdPrv
num_ftrs = ['MSSubClass','LotFrontage','LotArea','OverallQual','OverallCond'
            'MasVnrArea','BsmtFinSF1','BsmtFinSF2','BsmtUnfSF','TotalBsmtSF
            'LowQualFinSF','GrLivArea','BsmtFullBath','BsmtHalfBath','FullB
            'KitchenAbvGr','TotRmsAbvGrd','Fireplaces','GarageYrBlt','Garag
            'OpenPorchSF','EnclosedPorch','3SsnPorch','ScreenPorch','PoolAr
```

In [7]:
```
# preprocess with pipeline and columntransformer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import OrdinalEncoder
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.impute import SimpleImputer
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.ensemble import RandomForestRegressor

random_state = 42

# one-hot encoder
# We need to replace the NaN with a string first!
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='constant',fill_value='missing')),
    ('onehot', OneHotEncoder(sparse_output=False,handle_unknown='ignore'))])

# ordinal encoder
# We need to replace the NaN with a string first!
ordinal_transformer = Pipeline(steps=[
    ('imputer2', SimpleImputer(strategy='constant',fill_value='NA')),
    ('ordinal', OrdinalEncoder(categories = ordinal_cats))])

# standard scaler
numeric_transformer = Pipeline(steps=[
    ('scaler', StandardScaler())])

# collect the transformers
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, num_ftrs),
        ('cat', categorical_transformer, cat_ftrs),
        ('ord', ordinal_transformer, ordinal_ftrs)])
```

In [8]:

```
# fit_transform the training set
X_prep = preprocessor.fit_transform(X_train)
# the feature names after fit
feature_names = preprocessor.get_feature_names_out()

# you can convert the numpy array back to a data frame with the feature name
df_train = pd.DataFrame(data=X_prep,columns=feature_names)
print(df_train.shape)

# transform the CV
df_CV = preprocessor.transform(X_CV)
df_CV = pd.DataFrame(data=df_CV,columns = feature_names)
print(df_CV.shape)

# transform the test
df_test = preprocessor.transform(X_test)
df_test = pd.DataFrame(data=df_test,columns = feature_names)
print(df_test.shape)
print(feature_names)
```

```
(876, 222)
(292, 222)
(292, 222)
['num__MSSubClass' 'num__LotFrontage' 'num__LotArea' 'num__OverallQual'
 'num__OverallCond' 'num__YearBuilt' 'num__YearRemodAdd' 'num__MasVnrArea'
 'num__BsmtFinSF1' 'num__BsmtFinSF2' 'num__BsmtUnfSF' 'num__TotalBsmtSF'
 'num__1stFlrSF' 'num__2ndFlrSF' 'num__LowQualFinSF' 'num__GrLivArea'
 'num__BsmtFullBath' 'num__BsmtHalfBath' 'num__FullBath' 'num__HalfBath'
 'num__BedroomAbvGr' 'num__KitchenAbvGr' 'num__TotRmsAbvGrd'
 'num__Fireplaces' 'num__GarageYrBlt' 'num__GarageCars' 'num__GarageArea'
 'num__WoodDeckSF' 'num__OpenPorchSF' 'num__EnclosedPorch'
 'num__3SsnPorch' 'num__ScreenPorch' 'num__PoolArea' 'num__MiscVal'
 'num__MoSold' 'num__YrSold' 'cat__MSZoning_C (all)' 'cat__MSZoning_FV'
 'cat__MSZoning_RH' 'cat__MSZoning_RL' 'cat__MSZoning_RM'
 'cat__Street_Grvl' 'cat__Street_Pave' 'cat__Alley_Grvl' 'cat__Alley_Pave'
 'cat__Alley_missing' 'cat__LandContour_Bnk' 'cat__LandContour_HLS'
 'cat__LandContour_Low' 'cat__LandContour_Lvl' 'cat__LotConfig_Corner'
 'cat__LotConfig_CulDSac' 'cat__LotConfig_FR2' 'cat__LotConfig_FR3'
 'cat__LotConfig_Inside' 'cat__Neighborhood_Blmngtn'
 'cat__Neighborhood_Blueste' 'cat__Neighborhood_BrDale'
 'cat__Neighborhood_BrkSide' 'cat__Neighborhood_ClearCr'
 'cat__Neighborhood_CollgCr' 'cat__Neighborhood_Crawfor'
 'cat__Neighborhood_Edwards' 'cat__Neighborhood_Gilbert'
 'cat__Neighborhood_IDOTRR' 'cat__Neighborhood_MeadowV'
 'cat__Neighborhood_Mitchel' 'cat__Neighborhood_NAmes'
 'cat__Neighborhood_NPkVill' 'cat__Neighborhood_NWAmes'
 'cat__Neighborhood_NoRidge' 'cat__Neighborhood_NridgHt'
 'cat__Neighborhood_OldTown' 'cat__Neighborhood_SWISU'
 'cat__Neighborhood_Sawyer' 'cat__Neighborhood_SawyerW'
 'cat__Neighborhood_Somerst' 'cat__Neighborhood_StoneBr'
 'cat__Neighborhood_Timber' 'cat__Neighborhood_Veenker'
 'cat__Condition1_Artery' 'cat__Condition1_Feedr' 'cat__Condition1_Norm'
 'cat__Condition1_PosA' 'cat__Condition1_PosN' 'cat__Condition1_RRAe'
 'cat__Condition1_RRAn' 'cat__Condition1_RRNe' 'cat__Condition1_RRNn'
 'cat__Condition2_Artery' 'cat__Condition2_Feedr' 'cat__Condition2_Norm'
 'cat__Condition2_PosN' 'cat__Condition2_RRAe' 'cat__Condition2_RRAn'
 'cat__BldgType_1Fam' 'cat__BldgType_2fmCon' 'cat__BldgType_Duplex'
 'cat__BldgType_Twnhs' 'cat__BldgType_TwnhsE' 'cat__HouseStyle_1.5Fin'
 'cat__HouseStyle_1.5Unf' 'cat__HouseStyle_1Story'
 'cat__HouseStyle_2.5Fin' 'cat__HouseStyle_2.5Unf'
 'cat__HouseStyle_2Story' 'cat__HouseStyle_SFoyer' 'cat__HouseStyle_SLvl'
 'cat__RoofStyle_Flat' 'cat__RoofStyle_Gable' 'cat__RoofStyle_Gambrel'
 'cat__RoofStyle_Hip' 'cat__RoofStyle_Mansard' 'cat__RoofStyle_Shed'
 'cat__RoofMatl_ClyTile' 'cat__RoofMatl_CompShg' 'cat__RoofMatl_Metal'
 'cat__RoofMatl_Roll' 'cat__RoofMatl_Tar&Grv' 'cat__RoofMatl_WdShake'
 'cat__RoofMatl_WdShngl' 'cat__Exterior1st_AsbShng'
 'cat__Exterior1st_AsphShn' 'cat__Exterior1st_BrkComm'
 'cat__Exterior1st_BrkFace' 'cat__Exterior1st_CBlock'
 'cat__Exterior1st_CemntBd' 'cat__Exterior1st_HdBoard'
 'cat__Exterior1st_MetalSd' 'cat__Exterior1st_Plywood'
 'cat__Exterior1st_Stone' 'cat__Exterior1st_Stucco'
 'cat__Exterior1st_VinylSd' 'cat__Exterior1st_Wd Sdng'
 'cat__Exterior1st_WdShing' 'cat__Exterior2nd_AsbShng'
 'cat__Exterior2nd_AsphShn' 'cat__Exterior2nd_Brk Cmn'
 'cat__Exterior2nd_BrkFace' 'cat__Exterior2nd_CBlock'
 'cat__Exterior2nd_CmentBd' 'cat__Exterior2nd_HdBoard'
```

```
'cat__Exterior2nd_ImStucc' 'cat__Exterior2nd_MetalSd'
'cat__Exterior2nd_Other' 'cat__Exterior2nd_Plywood'
'cat__Exterior2nd_Stone' 'cat__Exterior2nd_Stucco'
'cat__Exterior2nd_VinylSd' 'cat__Exterior2nd_Wd Sdng'
'cat__Exterior2nd_Wd Shng' 'cat__MasVnrType_BrkCmn'
'cat__MasVnrType_BrkFace' 'cat__MasVnrType_Stone'
'cat__MasVnrType_missing' 'cat__Foundation_BrkTil'
'cat__Foundation_CBlock' 'cat__Foundation_PConc' 'cat__Foundation_Slab'
'cat__Foundation_Stone' 'cat__Foundation_Wood' 'cat__Heating_Floor'
'cat__Heating_GasA' 'cat__Heating_GasW' 'cat__Heating_Grav'
'cat__Heating_OthW' 'cat__Heating_Wall' 'cat__CentralAir_N'
'cat__CentralAir_Y' 'cat__Electrical_FuseA' 'cat__Electrical_FuseF'
'cat__Electrical_FuseP' 'cat__Electrical_SBrkr' 'cat__Electrical_missing'
'cat__GarageType_2Types' 'cat__GarageType_Attchd'
'cat__GarageType_Basment' 'cat__GarageType_BuiltIn'
'cat__GarageType_CarPort' 'cat__GarageType_Detchd'
'cat__GarageType_missing' 'cat__PavedDrive_N' 'cat__PavedDrive_P'
'cat__PavedDrive_Y' 'cat__MiscFeature_Gar2' 'cat__MiscFeature_Shed'
'cat__MiscFeature_TenC' 'cat__MiscFeature_missing' 'cat__SaleType_COD'
'cat__SaleType_CWD' 'cat__SaleType_Con' 'cat__SaleType_ConLD'
'cat__SaleType_ConLI' 'cat__SaleType_ConLw' 'cat__SaleType_New'
'cat__SaleType_Oth' 'cat__SaleType_WD' 'cat__SaleCondition_Abnorml'
'cat__SaleCondition_AdjLand' 'cat__SaleCondition_Alloca'
'cat__SaleCondition_Family' 'cat__SaleCondition_Normal'
'cat__SaleCondition_Partial' 'ord__LotShape' 'ord__Utilities'
'ord__LandSlope' 'ord__ExterQual' 'ord__ExterCond' 'ord__BsmtQual'
'ord__BsmtCond' 'ord__BsmtExposure' 'ord__BsmtFinType1'
'ord__BsmtFinType2' 'ord__HeatingQC' 'ord__KitchenQual' 'ord__Functional'
'ord__FireplaceQu' 'ord__GarageFinish' 'ord__GarageQual'
'ord__GarageCond' 'ord__PoolQC' 'ord__Fence']
```

In [9]:
```
print('data dimensions:',df_train.shape)
perc_missing_per_ftr = df_train.isnull().sum(axis=0)/df_train.shape[0]
print('fraction of missing values in features:')
print(perc_missing_per_ftr[perc_missing_per_ftr > 0])
print('data types of the features with missing values:')
print(df_train[perc_missing_per_ftr[perc_missing_per_ftr > 0].index].dtypes)
frac_missing = sum(df_train.isnull().sum(axis=1)!=0)/df_train.shape[0]
print('fraction of points with missing values:',frac_missing)
```

```
data dimensions: (876, 222)
fraction of missing values in features:
num__LotFrontage    0.190639
num__MasVnrArea     0.002283
num__GarageYrBlt    0.052511
dtype: float64
data types of the features with missing values:
num__LotFrontage    float64
num__MasVnrArea     float64
num__GarageYrBlt    float64
dtype: object
fraction of points with missing values: 0.23972602739726026
```

Quiz 1

The gender feature below contains missing values. Please explain how you would encode it and would be the output of the encoder. Do not write code. The goal of this quiz is to test your conceptual understanding so write text and the output array.

gender = ['Male', 'Female', 'Male', NaN, NaN, 'Female']

By the end of this lecture, you will be able to

- evaluate simple approaches to handle missing values: drop rows and columns
- handle missing values in categorical and ordinal features in preprocessing
- **apply imputation techniques on continuous features and evaluate drawbacks**

Continuous feature: sklearn's SimpleImputer

- Imputation means you infer the missing values from the known part of the data
- sklearn's SimpleImputer can do mean and median imputation
  - the missing values are replaced with the mean or median of the non-missing values
- A BAD IDEA!
  - mean or median imputation changes the statistical properties of the feature
  - specifically, it decreases the variance

In [26]:
```
import matplotlib.pyplot as plt

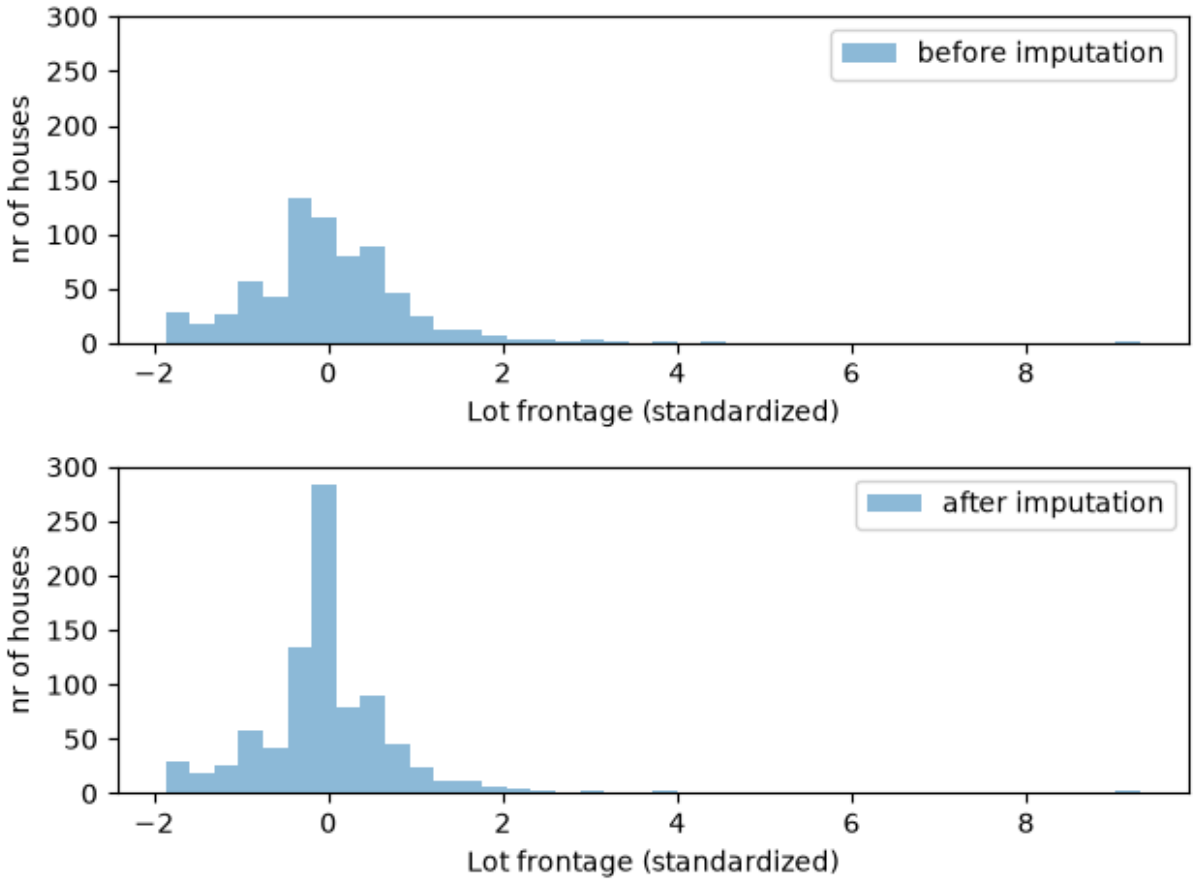
si = SimpleImputer(strategy='mean')
X_lot = si.fit_transform(df_train[['num__LotFrontage']])

plt.subplot(2, 1, 1)
df_train['num__LotFrontage'].hist(bins=40,label = 'before imputation',alpha=
plt.xlabel('Lot frontage (standardized)')
plt.ylabel('nr of houses')
plt.ylim(0,300)
plt.legend()
plt.tight_layout()

plt.subplot(2, 1, 2)
plt.hist(X_lot,bins=40,label='after imputation',alpha=0.5)
plt.xlabel('Lot frontage (standardized)')
plt.ylabel('nr of houses')
plt.ylim(0,300)
plt.legend()
plt.tight_layout()

plt.show()

print('std before imputation:',np.std(df_train['num__LotFrontage']))
print('std after imputation:',np.std(X_lot))
```




```
std before imputation: 1.0
std after imputation: 0.8996447802291788
```

Multivariate Imputation

- models each feature with missing values as a function of other features, and uses that estimate for imputation
  - at each step, a feature is designated as target variable and the other feature columns are treated as feature matrix X
  - a regressor is trained on (X, y) for known y
  - then, the regressor is used to predict the missing values of y
- in the ML pipeline:
  - create n imputed datasets
  - run all of them through the ML pipeline
  - generate n test scores
  - the uncertainty in the test scores is due to the uncertainty in imputation
- paper here

sklearn's IterativeImputer

- reproducability is important!
- we can't measure how accurate the imputed values are

- but we can measure how much uncertainty they add to the model's predictions!
  - use a non-deterministic estimator like a decision tree
  - loop through 3-10 random states and create 3-10 imputed datasets
  - measure the mean and std of the test score
  - uncertainty might be confounded with other sources of uncertainty due to random splitting and non-deterministic ML models

In [11]:
```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.ensemble import RandomForestRegressor

print(df_train[['num__LotFrontage','num__MasVnrArea','num__GarageYrBlt']].he

imputer = IterativeImputer(estimator = RandomForestRegressor(n_estimators=1,
X_impute = imputer.fit_transform(df_train)
df_train_imp = pd.DataFrame(data=X_impute, columns = df_train.columns)

print(df_train_imp[['num__LotFrontage','num__MasVnrArea','num__GarageYrBlt']

df_CV_imp = pd.DataFrame(data=imputer.transform(df_CV), columns = df_train.c
df_test_imp = pd.DataFrame(data=imputer.transform(df_test), columns = df_tra
```

```
   num__LotFrontage  num__MasVnrArea  num__GarageYrBlt
0         -0.372911        -0.606613         -2.126354
1         -0.678966        -0.606613         -1.927257
2               NaN         0.706580          0.063711
3          0.200943        -0.606613          0.422085
4         -0.066855        -0.606613          1.138834
   num__LotFrontage  num__MasVnrArea  num__GarageYrBlt
0         -0.372911        -0.606613         -2.126354
1         -0.678966        -0.606613         -1.927257
2          0.774798         0.706580          0.063711
3          0.200943        -0.606613          0.422085
4         -0.066855        -0.606613          1.138834
```

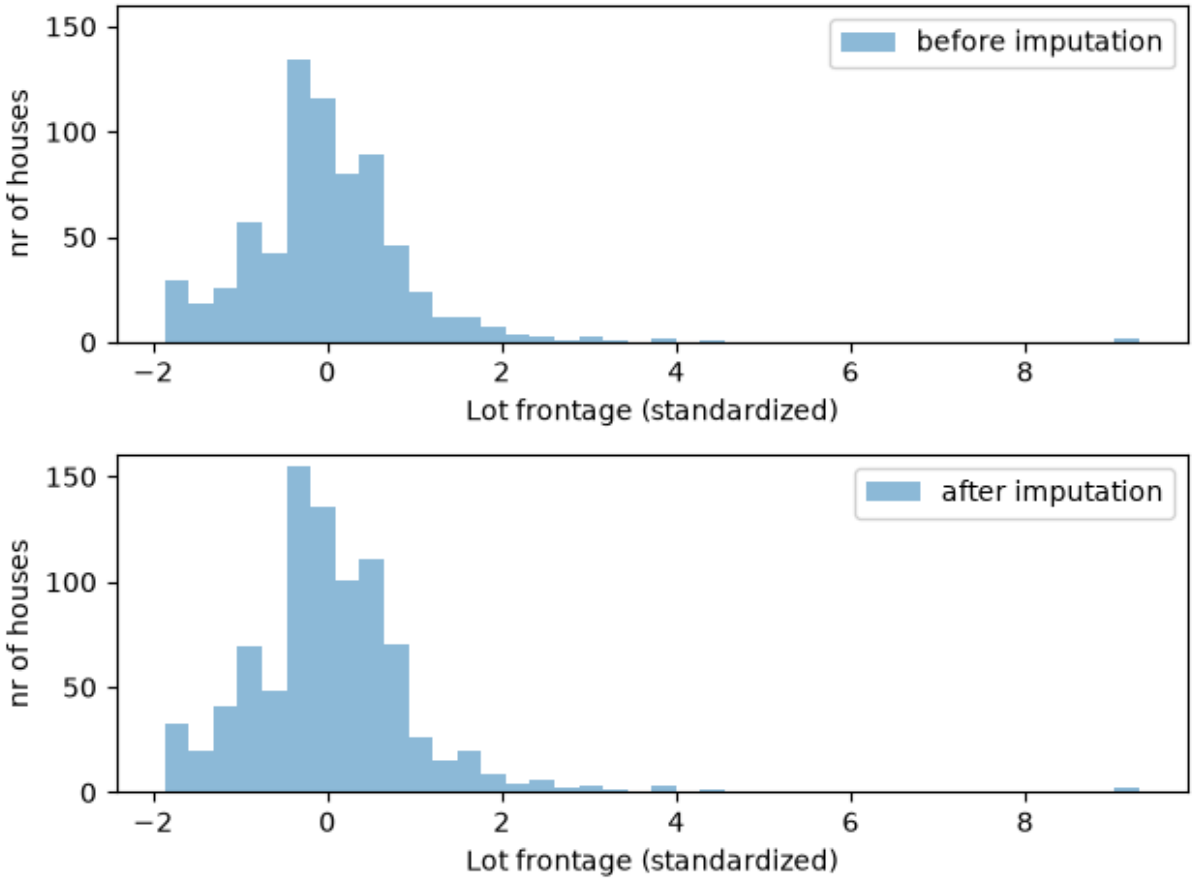
In [29]:
```
import matplotlib.pyplot as plt

plt.subplot(2, 1, 1)
df_train['num__LotFrontage'].hist(bins=40,label = 'before imputation',alpha=
plt.xlabel('Lot frontage (standardized)')
plt.ylabel('nr of houses')
plt.ylim(0,160)
plt.legend()
plt.tight_layout()

plt.subplot(2, 1, 2)
plt.hist(df_train_imp['num__LotFrontage'],bins=40,label='after imputation',a
plt.xlabel('Lot frontage (standardized)')
plt.ylabel('nr of houses')
plt.ylim(0,160)
plt.legend()
plt.tight_layout()

plt.show()
```

```
print('std before imputation:',np.std(df_train['num__LotFrontage']))
print('std after imputation:',np.std(df_train_imp['num__LotFrontage']))
```




```
std before imputation: 1.0
std after imputation: 0.9909703505115883
```

Does it make sense to impute?

- GarageYearBuilt should definitely not be imputed because a missing value indicates no garage on the property
- What does it mean to make up a year of construction for a garage that does not exist?

Mudcard